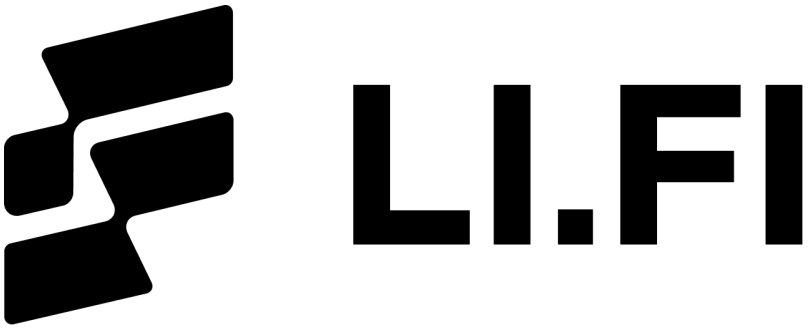

LI.FI Security Review

OutputValidator(v1.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

June 3, 2026

Contents

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Periphery/OutputValidator.sol(v1.0.0)

4 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

High leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.

Medium global losses <10% or losses to only a subset of users, but still unacceptable.

Low losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 4 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 7 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	c1caf50a
Audit Timeline	June 03, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Gas Optimizations	0
Informational	6
Total Issues	**7**

6 Findings

6.1 Low Risk

6.1.1 No event emitted on slippage capture

Context: OutputValidator.sol#L27, OutputValidator.sol#L80

Description: `validateNativeOutput` and `validateERC20Output` move funds to `validationWalletAddress` without emitting any event. The only observable event in the contract is the inherited `TokensWithdrawn (owner recovery)`, which does not cover the contract's primary action.

For a contract whose sole purpose is routing captured slippage to a wallet, this lack of an event makes off-chain accounting, monitoring, and incident response harder, and leaves no on-chain record distinguishing a capture from an ordinary transfer.

Recommendation: Emit a dedicated event on each successful capture covering both the native and ERC20 paths.

LI.FI: Agreed, fixed. We added a dedicated OutputValidated(address indexed token, address indexed validationWallet, uint256 excessAmount) event and emit it on every capture, on both the native and the ERC20 path (token is the null address for native). This matches the convention of our sibling periphery contracts such as FeeForwarder and FeeCollector, and it gives the native path an explicit log, which it did not have before. We also added test assertions (vm.expectEmit) for the event on both paths. Done in commit ccb0e6c7.

Researcher: Verified fix.

6.2 Informational

6.2.1 Use `IERC20` interface instead of importing full Solmate `ERC20` implementation

Context: OutputValidator.sol#L5

Description: `OutputValidator` imports the full Solmate ERC20 implementation only to call `balanceOf`:

```
import { ERC20 } from "solmate/tokens/ERC20.sol";
```

The contract then uses it here:

```
uint256 outputAmount = ERC20(tokenAddress).balanceOf(msg.sender);
```

This does not require the full ERC20 implementation type. The code already uses OpenZeppelin's `IERC20` interface for ERC20 balance and allowance interactions, for example in LibAsset. Using `IERC20` would be lighter and more consistent with the surrounding codebase.

Recommendation: Replace the Solmate ERC20 import with OpenZeppelin's IERC20 interface:

```
import { LibAsset, IERC20 } from "../Libraries/LibAsset.sol";
```

LI.FI: Agreed, fixed. We replaced the full solmate ERC20 import with IERC20 from LibAsset, which is consistent with the rest of the codebase. One small note: this is a code-consistency change only and has no bytecode or gas impact, since the balanceOf call selector is identical either way. Done in commit ccb0e6c7

Researcher: Verified fix.

6.2.2 Native validation can perform unnecessary zero-value external calls

Context: OutputValidator.sol#L27

Description: `validateNativeOutput` always calls `LibAsset.transferAsset` in some branches, even when the transferred amount is 0.

For example, if `msg.value == 0` and the caller's native balance is greater than `expectedAmount`, the function enters the excess branch:

```
if (excessAmount >= msg.value) {
      LibAsset.transferAsset(
          LibAsset.NULL_ADDRESS,
          payable(validationWalletAddress),
          msg.value
      );
  }
```

Because `msg.value` is 0, this performs a zero-value ETH call to `validationWalletAddress`.

This wastes gas even though no value is being transferred. The same pattern can also occur on refund paths where the refund amount is 0.

Recommendation: Skip native transfers when the amount is zero. For example, only call `LibAsset.transferAsset` when the computed amount is greater than zero.

LI.FI: Agreed, fixed. We added amount > 0 guards around the native transfers so the contract no longer makes zero-value native calls when msg.value == 0. This also removes the small griefing surface where a contract recipient could revert in its receive/fallback on a zero-value call. We added a test (test_validateOutputNativeWithZeroMsgValueSkipsTransfer) using a revert-on-receive wallet, which would fail without the guard. Done in commit ccb0e6c7.

Researcher: Verified fix.

6.2.3 Native output accounting depends on caller balance but payouts are capped by msg.value

Context: OutputValidator.sol#L37

Description: `validateNativeOutput` calculates `outputAmount` using both the caller's current ETH balance and the ETH sent to `OutputValidator`:

```
uint256 outputAmount = address(msg.sender).balance + msg.value;
```

This can be confusing because the function appears to account for the caller's full native balance, but it can only transfer ETH that was sent in the current call. All transfers are capped by msg.value; pre-existing ETH held by the caller is never transferred by `OutputValidator`.

In the current integration pattern, the Diamond sends only the intended excess amount to `OutputValidator` and calls the function with `expectedAmount = 0`. In that case, the `msg.sender.balance` term does not affect the outcome: the full `msg.value` is forwarded to the validation wallet.

This is not a direct fund-loss issue, but the accounting model is easy to misread and could lead to incorrect future integrations.

Recommendation: Document the intended native usage explicitly:

- OutputValidator cannot transfer pre-existing ETH from the caller.
- Native payouts are capped by msg.value.
- If the intended integration is "send only the excess amount to OutputValidator," then `expectedAmount` should be 0 and the full `msg.value` is forwarded.

Optionally simplify the native path to avoid reading `msg.sender.balance` if full caller-balance accounting is not required.

LI.FI: Agreed, this is a documentation point and we addressed it that way.

You are right that payouts are always capped by msg.value and that pre-existing native held by the caller is never moved. We confirmed the msg.sender.balance term can never cause more than msg.value to leave the contract, so there is no fund-loss risk. We added NatSpec to validateNativeOutput describing the cap and the intended integration (forward only the excess as msg.value, with expectedAmount == 0). We kept the balance term as-is so the accounting stays correct for non-zero expectedAmount uses. Documented in commit ccb0e6c7.

Researcher: Verified fix.

6.2.4 Redundant cast of `msg.sender` to address

Context: OutputValidator.sol#L37

Description: In `validateNativeOutput`, the balance is read via `address(msg.sender).balance`. `msg.sender` is already of type address, so the explicit `address(msg.sender)` cast is redundant.

Recommendation: Drop the cast and use `msg.sender.balance`.

LI.FI: Agreed, fixed. We removed the redundant address(msg.sender) cast and now use msg.sender.balance. Done in commit ccb0e6c7.

Researcher: Verified fix.

6.2.5 Zero-address validation of validationWalletAddress is conditional on excess existing

Context: OutputValidator.sol#L27, OutputValidator.sol#L80

Description: Neither `validateNativeOutput` nor `validateERC20Output` checks `validationWalletAddress` up front.

The only zero-address guard lives inside `LibAsset`, which is reached exclusively on the excess-transfer path. As a result, calls with no excess to forward succeed silently with `validationWalletAddress == address(0)`, while the with-excess paths revert `InvalidReceiver`.

Input validity thus depends on runtime balances rather than the input itself, and is inconsistent.

Recommendation: Validate `validationWalletAddress != address(0)` once at the top of both functions (revert `InvalidReceiver`), making the check deterministic and independent of whether excess happens to exist. Or consider documenting this asymmetry.

LI.FI: Agreed on the observation, and we decided to keep the current behaviour.

The zero-address guard already exists in LibAsset (transferNativeAsset and transferFromERC20 both revert with InvalidReceiver on address(0)). It only runs on the transfer path, so when there is no excess the call simply does nothing. The only case that "wrongly" succeeds moves zero funds, so there is no risk. Adding an upfront check would cost gas on every happy-path call for no real benefit, which is why we intentionally skip input validation here. We consider this a harmless asymmetry rather than a bug.

Researcher: Acknowledged.

6.2.6 Whitelisted OutputValidator can skim unrelated ERC20 balances from the Diamond

Context: OutputValidator.sol#L37, OutputValidator.sol#L90

Description: `OutputValidator.validateERC20Output` assumes this invariant: Diamond's current balance of `tokenAddress = output of the current swap`

```
uint256 outputAmount = ERC20(tokenAddress).balanceOf(msg.sender);
```

This assumption is unsafe. The function reads the Diamond's full ERC20 balance because, when called from a LI.FI swap route, `msg.sender` is the Diamond. It then sends everything above `expectedAmount` to `validationWalletAddress`.

Both `tokenAddress` and `validationWalletAddress` come from user-controlled calldata. They are not checked against the actual swap output token or against a trusted validation wallet.

This means a caller can create a route where `OutputValidator` is used as one whitelisted swap step and point it at a token the Diamond already holds for another reason, such as dust, stuck funds, or funds temporarily present during another route.

For example:

1. The Diamond already holds 100 DAI.
2. A user submits a route that calls validateERC20Output(DAI, 0, attackerWallet).
3. OutputValidator reads the Diamond's full DAI balance: 100 DAI.
4. Since expectedAmount = 0, it transfers all 100 DAI to attackerWallet.

This is not positive slippage from the user's swap. It is unrelated Diamond-held balance being misclassified as excess output and transferred away.

Recommendation: This is a known-risk in many facets, hence document this behavior to notify users / integrators.

Additionally, if you decide to fix this issue, do not let `OutputValidator` operate on arbitrary full Diamond balances. Instead, prefer delta-based accounting.

LIFI: Thanks for the detailed write-up. We agree the mechanic is real, but we see the severity as lower than Medium, and here is why.

For validateERC20Output to pull tokens, the Diamond must first approve OutputValidator for that token. That approval only happens inside the same route, in the same transaction, so an attacker cannot reach into another user's transaction — routes are atomic. The only funds that can actually be touched are balances that already sit in the Diamond before the call (dust or stuck funds). By design the Diamond does not custody funds between transactions, so any such balance is incidental and is already sweepable across the protocol today.

This is also not a new pattern. GenericSwapFacetV3 reads the Diamond's full balance and sends it out the same way (IERC20(receivingAssetId).balanceOf(address(this)) $\rightarrow$ full transfer to the receiver, plus _returnPositiveSlippageERC20). OutputValidator follows the established positive-slippage model.

We will not switch to delta-based accounting, because forwarding the full excess balance is exactly what this contract is for. Instead we documented the behaviour and the "only as a whitelisted swap step" assumption directly in the contract NatSpec. Done in commit ccb0e6c7.

Researcher: Acknowledged. Downgraded issue from `medium` to `informational` severity.